[redacted]

School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	[redacted]
Student Name:	Maël Conejos Crusson

Module Information

Module Title:	Fundamentals of AI & Data Science
Module Code:	[redacted]

Assessment Information

Assessment Title:	Project

Declaration

☑ I confirm that I have read and understood the University's Academic Integrity Policy.

[redacted]

☑ I have completed the TU Dublin AI Driving License theory test.

[redacted]

☑ I declare that this assessment submission is my own work.

☑ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	[redacted]
Date:	17/04/2026

[redacted]

[redacted]

Fundamentals of AI & Data Science Project

[redacted]

Maël Conejos Crusson
17/04/2026

[redacted]

Table of Contents

Executive Summary 3
Introduction 3
Data Understanding & Exploration 3
Methodology 7
Baseline Model 7
Improved Model 8
Results 12
Concerning Outliers 12
Improved DT: Outliers vs No Outliers 13
Improved KNN: Outliers vs No Outliers 13
Improved DT (No Outliers) vs Improved KNN (No Outliers) 14
The Official Results (Fig.14) 14
The Linear Regression Model (Fig. 16) 17
Conclusions & Reflections on Learning 18
Link to Python Notebook 18

Executive Summary

The purpose of this project is to use several models with different properties to identify malicious network traffic. I created several models to compare, showing that some models can distinguish between benign activity and potential attacks such as Brute-Force and Denial-of-Service better than others.

The project's results show the how important the context is in of preprocessing, such as keeping the outliers, which may indicate attacks. The classification models achieved more or less high accuracy, but the results reveal complications of class imbalance and the necessity to prioritise recall to ensure threats are not missed.

Introduction

Cybersecurity in a financial organisation is essential as they are targets for cyber threats. Any breach in a financial organisation would lead to big losses, along with fines and bad reputation. It is important to detect cyber threats before they become dangerous and ruin the integrity of a business.

This project's objective is to use AI and Data Science to identify patterns of malicious activity in network traffic. The focus is to detect attacks, such as Brute-Force attempts and Denial-of-Service attempts. Analysing features like 'login_failure_rate' and 'data_transfer_ratio' (I created both, and they will be explained later) help distinguish normal user behaviour compared to actual breaches.

Data Understanding & Exploration

The target feature of this dataset is 'is_attack', where I'll use the dataset's features to predict whether an attack will occur or not. I encoded the target feature (1 for 'Attack' and 0 for 'Benign') into a feature called 'attack'.

In terms of statistics, looking at the numeric analysis (Fig. 1) and the histograms (Fig. 2), there are some patterns that stand out. The 'packet_rate' and 'bytes_sent' are right-skewed, with outliers indicating a max 1147.06 for packet rate (the mean is only 126.13) and a max 48,743.94 for bytes sent (the mean is only 5275.75). These spikes indicate that (in a cybersecurity context) there is the presence of Denial-of-Service attacks, where the goal is to overwhelm system resources with high volume traffic. Anomaly score is another feature, indicating that most sessions receive low scores (with a mean of 0.16 out of 1), with the amount decreasing as the anomaly score increases. This suggests that most

sessions are normal user behaviour, which puts into question if the target 'attack' is balanced or not.

```
# Numeric Columns Analysis #1
print(df.describe())
```

```
       login_attempts  failed_logins    bytes_sent  bytes_received  \
count     6000.000000    6000.000000   6000.000000     5580.000000
mean         1.972500       1.020667   5275.753584     5981.881281
std          1.415537       1.004527   3132.622483     1994.673822
min          0.000000       0.000000    100.000000      100.000000
25%          1.000000       0.000000   4009.388961     4662.928544
50%          2.000000       1.000000   5013.713497     5992.395457
75%          3.000000       2.000000   6060.269321     7333.109710
max          9.000000       7.000000  48743.941649    13019.206996

       packet_rate  connection_count  port_number  session_retries  \
count  6000.000000       6000.000000  6000.000000       6000.00000
mean    126.137459          4.963500   472.757000          1.01950
std      71.772746          2.268485  1033.630588          1.00081
min      23.362556          0.000000    21.000000          0.00000
25%      99.854757          3.000000    25.000000          0.00000
50%     120.528816          5.000000    80.000000          1.00000
75%     140.300293          6.000000   143.000000          2.00000
max    1147.060032         15.000000  3389.000000          6.00000

       signal_strength  anomaly_score  session_duration       attack
count      5580.000000    6000.000000       6000.000000  6000.000000
mean        -59.996008       0.283358         92.914416     0.253167
std           9.981340       0.159846         16.595032     0.434862
min         -94.085793       0.002210         32.736351     0.000000
25%         -66.789129       0.157360         81.428235     0.000000
50%         -60.054692       0.260576         92.878546     0.000000
75%         -53.439260       0.389571        104.194079     1.000000
max         -25.139054       0.824099        156.431365     1.000000
```

Figure 1: Numeric Columns Analysis #1

```
# Histograms
df.hist(figsize=(12, 8), bins=20, edgecolor="black")
plt.suptitle('Distributions of All Numeric Columns')
plt.tight_layout()
plt.show()
```

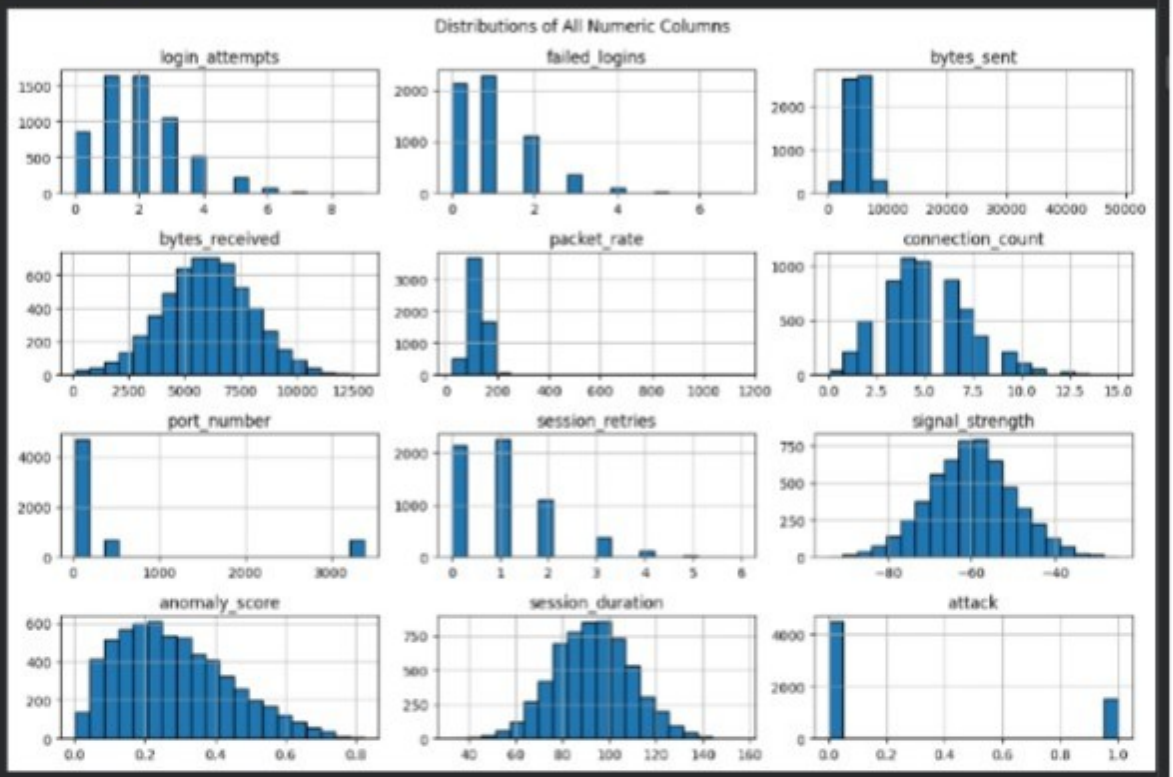


Figure 2: Histograms

In terms of behavioural patterns, the scatterplots (Fig. 3) show a connection between 'failed_logins' and 'login_attempts'. There's a correlation between these two variables that indicate Brute-Force attack patterns, where the high number of attempts correspond to the high failure rate when an attacker guesses credentials over and over again (anything above 2 login attempts will result in there being more failures on average than below 2).

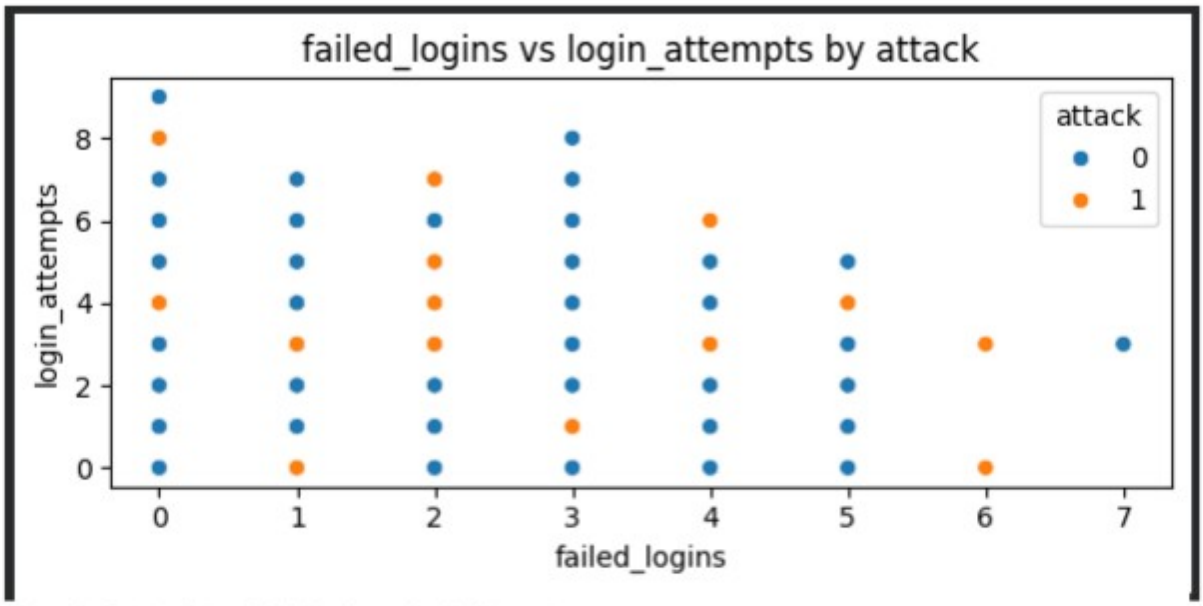


Figure 3: Scatterplots - Failed Logins vs Login Attempts

In terms of data quality, there are three features that contain 420 missing features each (7% of missing data) (Fig. 4): 'protocol', 'bytes received', and 'signal strength'. Columns had to be imputed with mean and mode to maintain integrity. Another feature indicating poor data quality is the imbalance of the target itself, resulting in an imbalanced dataset (Fig. 5): 74.4% (4,481 rows) of the data is 'Benign' traffic and only 25.3% (1,519 rows) of the data is 'Attack'. This imbalance leads to how low the recall scores are in the classification models as well as the high number of false negatives. This is terrible in a cybersecurity context since it means that a large number of attacks have not been detected by the models.

```
# Missing Values
missing = pd.DataFrame({
 'Missing Count': df.isnull().sum(),
 'Missing %': (df.isnull().sum() / len(df)) * 100
})
missing = missing[missing['Missing Count'] > 0].sort_values('Missing %', ascending=False)
print(missing)
```

```
                Missing Count  Missing %
protocol                  420        7.0
bytes_received            420        7.0
signal_strength           420        7.0
```

Figure 4: Missing Values

```
# Target Distribution
print(df['attack'].value_counts())
print(df['attack'].value_counts(normalize=True) * 100)
print("")

plt.figure(figsize=(6, 4))
sns.countplot(data=df, x='is_attack', palette='Set2')
plt.title('Distribution of Benign vs Attack')
plt.show()
```

```
attack
0    4481
1    1519
Name: count, dtype: int64
attack
0    74.683333
1    25.316667
Name: proportion, dtype: float64
```

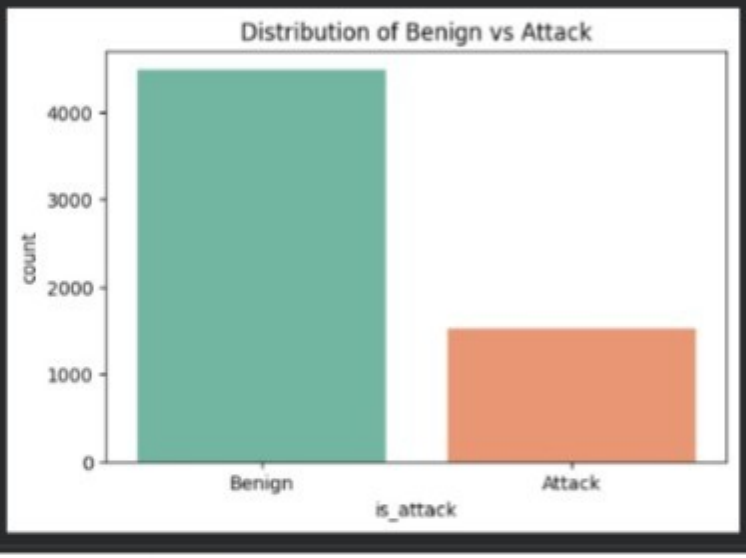


Figure 5: Target Distribution

Methodology

Baseline Model

I started with a baseline model for training on raw, dirty data (Fig. 6). Its purpose is to create a simple Decision Tree model as a performance benchmark. I purposefully didn't do the recommended approach to deal with data in this model.

```
# Create Baseline Dataset
df_baseline = df.copy()

# Dropping Useless Columns
df_baseline = df_baseline.drop(['session_id', 'is_attack'], axis=1)

# Numeric Features
num_features = ['failed_logins', 'packet_rate', 'anomaly_score', 'session_duration', 'connection_count', 'signal
X_baseline = df_baseline[num_features]
y_baseline = df_baseline['attack']

# Remove Missing Values
X_baseline_clean_rows = X_baseline.dropna()
y_baseline_clean_rows = y_baseline[X_baseline_clean_rows.index]

# Encoding
for col in X_baseline_clean_rows.select_dtypes(include='object').columns:
  le_protocol_baseline = LabelEncoder()
  X_baseline_clean_rows[col] = X_baseline_clean_rows[col].astype(str)
  X_baseline_clean_rows[col] = le_protocol_baseline.fit_transform(X_baseline_clean_rows[col])
```

Figure 6: Creating Baseline Model

So, I started by dropping two useless columns: 'session_id' and 'is_attack'. The 'session_id' column is useless since its only purpose is to identify each row. The 'is_attack' column is redundant since it is the original target, and I have created the new encoded target 'attack'.

I then selected specific numeric features for this dataset that I thought would be relevant in a cybersecurity context:

- 'failed_logins' where high values are indicators of Brute-Force attacks.
- 'packet_rate' which is useful for detecting DoS or flooding.
- 'anomaly_score' which holds weight since it was precalculated.
- 'session_duration' where very short or very long sessions usually deviate from the norm.
- 'connection_count' where high counts can indicate scanning.
- 'signal_strength' which is important if the attack involves proximity.

I then did the non-recommended approach to dealing with missing data: removing the rows entirely that contain missing data.

And I finally encoded everything using Label Encoder.

Improved Model

The Improved model follows the recommended approach to dealing with data, and specifically in this context (Fig. 7-8). I began by copying the original data frame onto this one.

```
# Create Cleaned Dataset
df_improved = df.copy()

# Standardise Categorical Columns
for col in df_improved.select_dtypes(include='object').columns:
  df_improved[col] = df_improved[col].str.lower().str.strip()
  print(f"Standardised column: {col}")

# Fix Wrong Values - [no categorical columns found with wrong values]

# Fix Port Values (they are numeric, but shouldn't be (for example, port 433 is higher than port 80, but is ont more important...))
# Convert port_number to string so it is treated as categorical
df_improved['port_number'] = df_improved['port_number'].astype(str)
# Map common ports to names for better readability in feature importance
port_mapping = {
    '80': 'HTTP', '443': 'HTTPS', '21': 'FTP', '22': 'SSH',
    '25': 'SMTP', '53': 'DNS', '110': 'POP3', '143': 'IMAP', '3389': 'RDP'
}
df_improved['port_number'] = df_improved['port_number'].replace(port_mapping)
print("\nConverted 'port_number' to categorical strings.")

# Handle Missing Values
# Over 40% - [no columns found with >40% missing data]
# 5-40% Missing
df_improved['protocol'] = df_improved['protocol'].fillna(df_improved['protocol'].mode()[0]) # categorical
df_improved['bytes_received'] = df_improved['bytes_received'].fillna(df_improved['bytes_received'].mean()) # symmetric
df_improved['signal_strength'] = df_improved['signal_strength'].fillna(df_improved['signal_strength'].mean()) # symmetric
print("\nImputed column with mode: 'protocol'")
print("Imputed column with mean: 'bytes_received'")
print("Imputed column with mean: 'signal_strength'")
# Impute with median - [no incomplete columns found with skewed data]
# Under 5% Missing - [no columns found with <5% missing data]
df_improved.dropna(inplace=True)
df_improved.reset_index(drop=True, inplace=True)

# Remove Outliers - [not removing outliers because they may be indicative of an attack (like unusually high packet rates)]

# Remove Columns with only one value - [no columns found with only one unique value]
```

Figure 7: Creating Improved Model (1)

```
# Remove Redundant Columns
redundant_cols = ['is_attack']
for col in redundant_cols:
  df_improved = df_improved.drop(col, axis = 1)
  print(f"\nRemoved redundant column: '{col}'")

# Remove Useless Columns
useless_cols = ['session_id']
for col in useless_cols:
  df_improved = df_improved.drop(col, axis = 1)
  print(f"\nRemoved useless column: '{col}'")

# New Features
df_improved['login_failure_rate'] = df_improved['failed_logins'] / (df_improved['login_attempts'] + 1)
print("\nCreated 'login_failure_rate'")
df_improved['data_transfer_ratio'] = df_improved['bytes_sent'] / (df_improved['bytes_received'] + 1)
print("Created 'data_transfer_ratio'")

# Ordinal Encoding 'privilege_level' as it has an order
df_improved[['privilege_level']] = OrdinalEncoder().fit_transform(df_improved[['privilege_level']])
print(f"\nEncoded (Ordinal) column: '{col}'")
# One-Hot Encoding Categorical Features (the categorical columns have no natural order)
cat_cols = df_improved.select_dtypes(include='object').columns.tolist()
df_improved = pd.get_dummies(df_improved, columns=cat_cols, drop_first=True)
print(f"Encoded (One-Hot) columns: {cat_cols}")

# Scale Numeric Features (Min-Max)
print("")
df_prepared = df_improved.copy()
# Split Before Scaling
X_prepared = df_prepared.drop('attack', axis=1)
y_prepared = df_prepared['attack']
X_train, X_test, y_train, y_test = train_test_split(
    X_prepared, y_prepared, test_size=0.2, random_state=42)
# Fit on Training Data & Transform Both
num_cols = X_train.select_dtypes(include='number').columns.tolist()
scaler = MinMaxScaler()
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])
print("Fitted Training Data & Transformed Training & Testing Data")
```

Figure 8: Creating Improved Model (2)

Concerning fixing inconsistencies, I standardised every categorical column to be lowercase and to not include spaces. This ensures that all words maintain the same format, connecting variables that had different cases and spaces (for example, 'udp' and 'UDP ' now both are combined into 'udp'). After analysing the output of the categorical columns (Fig. 9) I did not find any irregularities in the names of the variables, so, no wrong values had to be fixed.

```
# Categorical Columns Analysis
for col in df.select_dtypes(include='object').columns:
  print(df[col].value_counts())
  print(f"Unique values: {df[col].nunique()}")
  print("")
```

```
S000002    1
S000001    1
S000000    1
Name: count, Length: 6000, dtype: int64
Unique values: 6000

protocol
TCP     2532
UDP     1378
tcp      572
Udp      557
ICMP     541
Name: count, dtype: int64
Unique values: 5

device_type
Server         1562
IoT            1510
Workstation    1470
Mobile         1458
Name: count, dtype: int64
Unique values: 4

vpn_status
Off    4184
On     1816
Name: count, dtype: int64
Unique values: 2

region
```

Figure 9: Categorical Column Analysis

I then fixed the port values as they are numerical, which is a problem. Since the port numbers are numerical, this means than the higher values are more important than the lower values (for example, 21 for 'FTP' is automatically less important than 143 for 'IMAP', which is wrong). So, I mapped out the port numbers to their name and replaced the 'port_number' feature with the named variants. So, in the encoding they will be treated equally as separate values.

Concerning missing values, I handled them with the recommended approach: any column with over 40% missing data must be dropped, any columns between 5 and 40% missing data must be replaced by their averages, and anything below 5% must have their rows with missing data dropped. Luckily only three columns had missing data, and all were at 7%, so I could do them separately: the 'protocol' column had all its missing values replaced with mode, since the column is categorical, and I replaced the missing values of both the 'bytes_received' and 'signal_strength' columns with their mean. I didn't replace with

median since it is only recommended for columns containing skewed data, which both didn't have (as seen in the histograms in Fig. 2, both are mostly symmetrical).

Concerning outliers, I chose not to remove them since they may indicate an attack. For example, unusually high packet rates would've been removed from the data frame, whereas they may actually indicate Denial-of-Service attacks. However, results do vary a bit, which will be explained later.

Concerning the removal of useless attributes, as there were no columns with only one unique value, there were none to remove in that regard. However, as mentioned, there is one useless column 'session_id' and one redundant column 'is_attack' which were both removed.

Concerning attribute construction, I added two attributes. One is 'login_failure_rate', which calculates the failed logins divided by the login attempts (with +1 to avoid division by 0). This feature separates Brute-Force attack patterns from potential human errors (like typos). The other is 'data_transfer_ratio', which calculates the bytes sent divided by the bytes received (with +1). This ratio helps identify anomalies such as unusually high sent data (potential data exfiltration or DoS).

Concerning encoding of the categorical features, I chose two encoding types: Ordinal Encoding and One-Hot Encoding, I did not use Label Encoding. I used Ordinal Encoding for only one categorical column: 'privilege_level', as this is the only categorical column with a hierarchy ('guest' < 'user' < 'admin'). I then used One-Hot Encoding for every other categorical column as they didn't have an order to them, allowing the model to not assume false order between categories.

Concerning the scaling of numeric features, I normalised all numeric features to a 0-1 range, preventing high-value features (such as 'bytes_sent') from taking over lower-value features (such as 'failed_logins'). This was also necessary since I was using the KNN algorithm, which is distance-based.

Results

Concerning Outliers

I've tested an improved model with the outliers (outlier section in Fig. 10) and the same exact model without the outliers.

```python
# Remove Outliers - [USED]
def remove_outliers_iqr(df, column):
  Q1 = df[column].quantile(0.25)
  Q3 = df[column].quantile(0.75)
  IQR = Q3 - Q1
  lower = Q1 - 1.5 * IQR
  upper = Q3 + 1.5 * IQR
  before = len(df)
  df_clean = df[(df[column] >= lower) & (df[column] <= upper)]
  after = len(df_clean)
  print(f"{column}: removed {before - after} rows | bounds [{lower:.2f}, {upper:.2f}]")
  return df_clean
# Apply to all numeric columns
numeric_cols = df_no_outliers.select_dtypes(include='number').columns
for col in numeric_cols:
 df_no_outliers = remove_outliers_iqr(df_no_outliers, col)
df_no_outliers.reset_index(drop=True, inplace=True)
```

Figure 10: Outliers Section

Improved DT: Outliers vs No Outliers

As seen in the comparisons using the DT Classifier algorithm (Fig. 11), keeping the outliers makes the model more accurate (77.00% accuracy with outliers, 73.97% without). However, by removing the outliers the model has an improved precision (0.54 vs 0.36), as well as recall, and therefore f1-score. This means that removing outliers makes the model more stable for stealthy attacks, but it causes it to miss more obvious threats, which leads to the drop in accuracy.

```
Comparing Classifiers
    Metric  Improved (DT + Outliers)  Improved (DT - Outliers)  Improved (DT + Outliers) vs Improved (DT - Outliers)
  Accuracy                    0.7700                    0.7397                                               4.0926
 Precision                    0.3636                    0.5417                                             -32.8671
    Recall                    0.0449                    0.0453                                              -0.7779
  F1-Score                    0.0800                    0.0836                                              -4.3077

Key Performance Insights
- Improved (DT + Outliers) vs Improved (DT - Outliers):  +3.03% accuracy change
- The Improved DT (+ Outliers) has the best Accuracy.
- The Improved DT (- Outliers) has the best Precision.
- The Improved DT (- Outliers) has the best Recall.
- The Improved DT (- Outliers) is the best F1-Score.
```

Figure 11: DT Outliers vs No Outliers

Improved KNN: Outliers vs No Outliers

The results from the DT differ from the KNN Classifier results (Fig.12), as keeping the outliers appear to make the model have a higher accuracy, precision, recall, and f1-score, with a 101% increase in recall. This is because KNN relies on space proximity, and since attacks are statistically distant from normal traffic, removing outliers deletes the examples the KNN needs to define an attack.

```
Comparing Classifiers
   Metric  Improved (KNN + Outliers)  Improved (KNN - Outliers)  Improved (KNN + Outliers) vs Improved (KNN - Outliers)
 Accuracy                     0.7492                     0.7087                                                5.7136
Precision                     0.2344                     0.1667                                               40.6250
   Recall                     0.0562                     0.0279                                              101.5449
 F1-Score                     0.0906                     0.0478                                               89.7659

Key Performance Insights
- Improved (KNN + Outliers) vs Improved (KNN - Outliers):  +4.05% accuracy change
- The Improved KNN (+ Outliers) has the best Accuracy.
- The Improved KNN (+ Outliers) has the best Precision.
- The Improved KNN (+ Outliers) has the best Recall.
- The Improved KNN (+ Outliers) is the best F1-Score.
```

Figure 12: KNN Outliers vs No Outliers

Improved DT (No Outliers) vs Improved KNN (No Outliers)

Considering the previous results, when comparing both DT and KNN models, having removed their outliers, the output indicates that the DT model wins everywhere (Fig.13), specifically with precision (0.54 vs 0.16). A precision of 0.16 for the KNN means that every 100 alerts generated, 84 are false positives. So, the DT is more appropriate for real-world deployment, if outliers were removed.

```
Comparing Classifiers
   Metric  Improved (DT - Outliers)  Improved (KNN - Outliers)  Improved (DT - Outliers) vs Improved (KNN - Outliers)
 Accuracy                    0.7397                     0.7087                                                4.3814
Precision                    0.5417                     0.1667                                              225.0000
   Recall                    0.0453                     0.0279                                               62.5000
 F1-Score                    0.0836                     0.0478                                               75.0402

Key Performance Insights
- Improved (DT - Outliers) vs Improved (KNN - Outliers):  +3.11% accuracy change
- The Improved Decision Tree (- Outliers) has the best Accuracy.
- The Improved Decision Tree (- Outliers) has the best Precision.
- The Improved Decision Tree (- Outliers) has the best Recall.
- The Improved Decision Tree (- Outliers) is the best F1-Score.
```

Figure 13: DT No Outliers vs KNN No Outliers

For the official results, I decided to keep the outliers for both the DT and KNN models.

The Official Results (Fig.14)

```
Comparing Classifiers
   Metric  Baseline (DT)  Improved (DT)  Improved (KNN)  Improved DT vs Baseline DT (%)  Improved KNN vs Baseline DT (%)  Improved DT vs Improved KNN (%)
 Accuracy         0.7052         0.7380          0.7492                          0.8230                           -2.0099                           2.7084
Precision         0.5517         0.3636          0.2344                         -34.0000                          -57.5195                          55.1515
   Recall         0.1194         0.0169          0.0562                         -82.3596                          -52.9894                         -20.0000
 F1-Score         0.1963         0.0000          0.0906                         -59.2500                          -53.8333                         -11.7333

Key Performance Insights
- Improved DT vs Baseline DT:  +0.40% accuracy change
- Improved KNN vs Baseline DT: -1.61% accuracy change
- Improved DT vs Improved KNN: +2.08% accuracy difference

Excluding the Baseline
- The Improved Decision Tree has the best Accuracy.
- The Improved Decision Tree has the best Precision.
- The Improved KNN has the best Recall.
- The Improved KNN is the best F1 Score.
```

Figure 14: Model Results

The baseline DT model has maintained the highest precision score (0.55) and f1-score (0.19) compared to the improved models. This is explained by the feature selection bias, where the baseline used a hand-picked features (mentioned in the Methodology section),

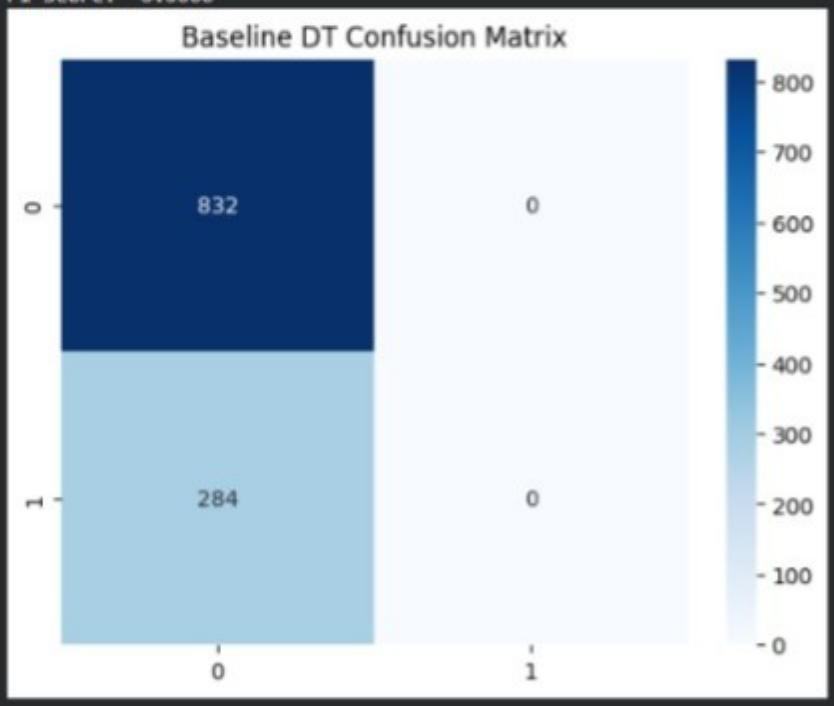
as they have a high correlation with attack signals. By adding all features in the improved models, noise was introduced. And in a small dataset, the models struggle to distinguish between relevant signals and random patterns in the new features. I've ran another test where I treated the baseline more or less the same as the improved models: I used all numeric columns as the features (the improved models also had the categorical columns). And the results were terrible (Fig. 15): there were only negatives (832 true negatives and 284 false negatives, 0 positives). This proves that the baseline on its own is not a good model to use, especially in a real-world scenario where false negatives are crucial to avoid.

```
Baseline Model Decision Tree Classifier
              precision    recall  f1-score   support

           0       0.75      1.00      0.85       832
           1       0.00      0.00      0.00       284

    accuracy                           0.75      1116
   macro avg       0.37      0.50      0.43      1116
weighted avg       0.56      0.75      0.64      1116


Accuracy:  0.7455
Precision: 0.0000
Recall:    0.0000
F1-Score:  0.0000
```




```
True Negatives:  832
False Positives: 0
False Negatives: 284
True Positives:  0
```

Figure 15: Noisy Baseline Results

This leads to the question if the improved models represent a more real-world scenario, and yes, they do. The improved models handle missing values and encode categorical data. And in a real scenario, models receive raw logs with strings and missing fields. In addition, using GridSearchCV ensures that it's not just picking the 'max_depth' that works for one specific split, it looks for a model that generalises better, even if the resulting metrics are lower.

Actually, comparing the improved DT (with outliers) to the improved KNN model (with outliers), the improved dataset contains many one-hot encoded categorical features, which Decision Trees handle naturally. However, KNN struggles with a lot of binary data as the distance between 0 and 1 in multiple categories becomes noisy ad loses meaning. Another note is that the DT model focuses on the most informative features (like 'failed_logins' or 'packet_rate') to make a split, but KNN treats every feature with equal importance (after scaling) which means that less relevant features can pull the neighbours away from the true classification, which leads to more false positive rates (49 vs 21), which leads to lower precision. KNN did however get a better recall and f1-score because it looks at the neighbours of points, so it's more likely to flag rare, isolated attack points that are near each other in a space, even if they don't follow a linear rule. So, it catches a few more attacks than the DT (recall 0.056 vs 0.044).

The takeaway is that the transition from baseline to improved models revealed a class imbalance issue that was hidden before. The low recall (which is below 12% for all models) is the most important part: no models are good for production because they miss the majority of threats. If one model had to be chosen, it would be the Improved Decision Tree Classifier with outliers.

The Linear Regression Model (Fig. 16)

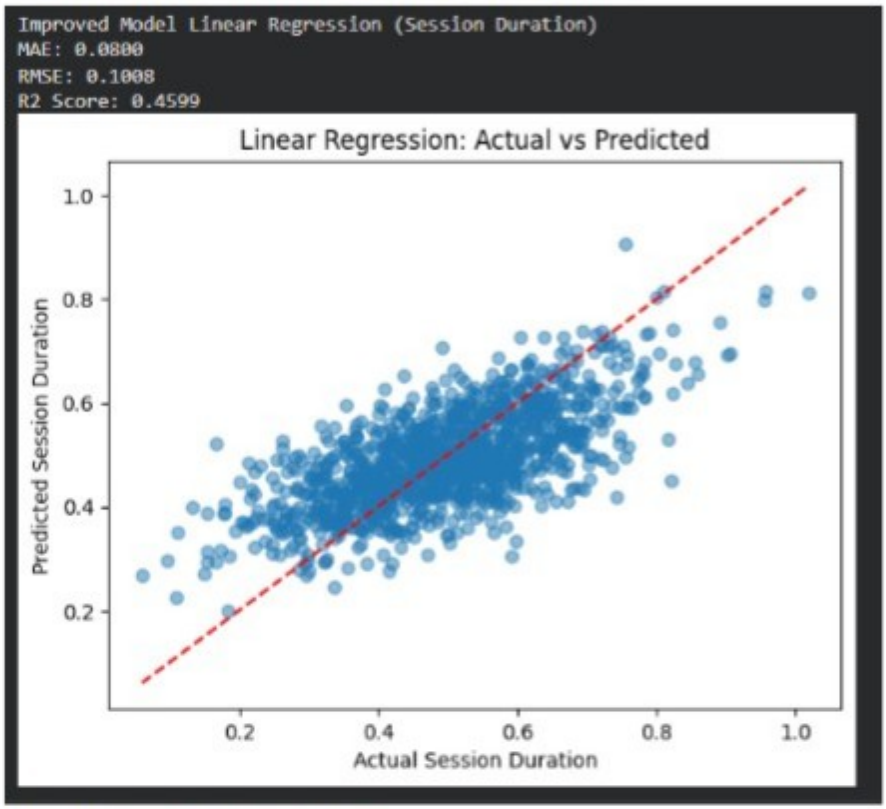


Figure 11: Linear Regression Results

Now focusing on the regression model, I used linear regression on the improved df. The purpose of this model is to predict the 'session_duration'.

The MAE (which is 0.0800) and RMSE (which is 0.1008) represent the average error in predicting the anomaly score. The RMSE of 0.10 shows that the model is fairly close to the true score, but misses by 10%, which is a relatively decent fit.

The R2 Score (0.4599) shows that there is roughly 46% of the variance in 'session_duration' that can be explained by the input features (like packet rate or login attempts). This does show that a relationship exists, but the 54% left of variance is likely because of factors not taken in the dataset or non-linear patterns.

In a cybersecurity context, if a session lasts quite longer or shorter than the model predicts, it may indicate a bot, data exfiltration, or an attack that stays connected to evade detection. The scatterplot showed that the red line which is the general trend suggests that as network activity (like bytes and packets) increase, the duration leans toward 'Benign' traffic, and deviations from this is where analysts generally focus.

Conclusions & Reflections on Learning

This project showed that the Decision Tree Classifier model (with outliers) was the best model for detecting attacks overall, even outperforming the KNN Classifier model which struggled with the encoded categorical data. But the most important finding was the impact of class imbalance: because the dataset consisted of a lot of benign traffic (over 74%), all classification models struggled from the low recall rates (under 12%). This means the even if the overall accuracy is good, the models missed most of the actual attacks, which proves they are not ready for a real-world scenario.

This project made me realise the reality of working with messy data and the importance of context. For example, while normal practice says you should remove extreme values, this project showed that keeping them was essential, as they might indicate attacks (for example, with unusually high packet rates). And that having a select amount of features to train and test the models on is far better than just using every attribute, as it can become noisy. And finally, making custom features like 'login_failure_rate' helps give the model more context, and relying on accuracy alone is misleading when dealing with imbalanced threats.

[redacted]